

EcoDrop Gravity Battery — Model and Simulation Documentation

Scope and Conventions

- **Language:** The implementation uses **MATLAB** throughout (scripts and functions under the `matlab/` folder). Python files may exist for prototyping, but the authoritative version for this project and the science fair presentation is the MATLAB codebase.
 - **Workspace:** The plan includes a **calibration validation step** using the physical build parameters (25 kg, ~2.7 m, 1:25 gearbox, BLDC) so simulated output can be tuned to match multimeter data.
 - **Diagrams:** System diagrams/CAD are in the attached PowerPoint. Use them as reference when implementing each system's mechanics and return mechanism.
 - **Motor reference:** Turnigy SK3-5065-236KV ($K_v = 236 \text{ RPM/V}$, $R_m = 0.019 \Omega$, $K_t \approx 0.0405 \text{ Nm/A}$). **Motor efficiency** from physical build calibration (spec: $\eta_{\text{motor}} = 0.82$), with $\pm 2\%$ perturbation per run.
-

1. Project Layout

```
gravity battery/
├─ matlab/
│   ├── config.m           # Constants: g, h, masses,  $\mu$ ,  $\eta_{\text{gear}}$ ,  $K_v$ ,  $R_m$ , bench
│   ├── physics_core.m     # Shared: PE_input, friction (rope, bearing), gearb
│   └─ systems/
│       ├── run_cycle_dual_weight.m    # System 1: two drums, counterweight, reversibl
│       ├── run_cycle_variable_cw.m    # System 2: variable counterweight, modular gen
│       ├── run_cycle_buoyancy.m       # System 3: pump, buoyant weight, water drag
│       └─ run_cycle_halbach.m         # System 4: linear motors, Halbach, auxiliary h
│   ├── run_simulation.m    # Orchestrator: 100 runs  $\times$  4 systems, perturbations
│   ├── metrics.m          # discharge efficiency, loss breakdown, net energy,
│   ├── plotting.m         # All graph sets (1A-1C, 2A-2B, 3A-3E), styling, PN
│   ├── calibration.m      # Physical-build validation (25 kg, ~2.7 m)  $\rightarrow$  tune
│   ├── main.m             # Entry: calibration check  $\rightarrow$  run_simulation  $\rightarrow$  table
│   └─ outputs/           # summary_table.csv and all figures
└─ python/               # (Optional) legacy Python prototype, not used for
```

Single-entry flow: `main.m` runs calibration (optional/skip if no data), then `run_simulation.m`, then `metrics.m` aggregation, then `plotting.m`, and writes CSV + console output.

2. Core Physics (Reusable)

config.m — Centralize: $g = 9.81$, $h = 3.0$, $m_{\text{primary}} = 50$, $PE_{\text{input}} = 1471.5 \text{ J}$; μ_{rope} , $\theta = \pi$; μ_{bearing} , r_{shaft} ; $\eta_{\text{gear_stage}} 0.94$, ratio 1:25 $\rightarrow \eta_{\text{gearbox}} = 0.94^2$; K_v , R_m , $\eta_{\text{motor_base}} = 0.82$; benchmark discharge efficiency lines 77.5% and 88.5%.

physics_core.m — Gravitational PE per cycle; rope tension efficiency; bearing torque; gearbox; motor/generator (Back-EMF, $P_{\text{elec}}/P_{\text{mech}}$); descent dynamics (ODE integration). Friction and motor efficiency accept optional scaling (e.g. $\pm 5\% \mu$, $\pm 2\% \eta$) for the 100-run and parameter-sensitivity analyses.

3. Four System Models

Each system returns: (1) **E_electrical_out** (discharge), (2) **E_consumed** (return/charge), (3) **loss breakdown** (rope, bearing, gearbox, motor, system-specific), (4) time series for one representative cycle.

System 1: Dual Weight (`run_cycle_dual_weight.m`)

- Two drums, opposite wrap; primary 50 kg, counterweight 40 kg. Net imbalance 10 kg drives generation. Discharge/return model; reversible gearbox 1:25.

System 2: Variable Counterweight (`run_cycle_variable_cw.m`)

- Primary 50 kg; counterweight 45–50 kg (e.g. $5 \times 1 \text{ kg}$ modules). Main + modular generators; E_{consumed} includes control system.

System 3: Buoyancy (`run_cycle_buoyancy.m`)

- Weight 50 kg, volume 0.06 m^3 ; water drag model; pump charge $\eta_{\text{pump}} = 0.70$.

System 4: Halbach (`run_cycle_halbach.m`)

- Linear motors, Halbach factor 1.4; auxiliary hoist and control; magnetic drag.
-

4. Run Protocol and Metrics

run_simulation.m — 100 runs per system; perturb $\eta_{\text{motor}} \pm 2\%$, friction $\pm 5\%$. Parameter sensitivity: motor loss scale 0.8–1.2 ($\pm 20\%$), system-specific scale 0.5–1.5; see `sensitivity_motor_system.m`.

metrics.m — Metric 1: Discharge efficiency ($E_{\text{out}}/PE_{\text{input}} \times 100$). Metric 2: Loss breakdown. Metric 3: Net energy per cycle. Metric 4: P_{mech} vs P_{elec} . Metric 5: Cumulative net (1–500). Metric 6: Parameter sensitivity heatmap. ANOVA on discharge efficiency and net energy (100 runs each).

5. Plotting and Output

plotting.m — Graph sets 1–3 (RTE, loss breakdown, net energy, cumulative, sensitivity). Outputs to `outputs/`. Master Summary Table and ANOVA to console and CSV.

6. Validation

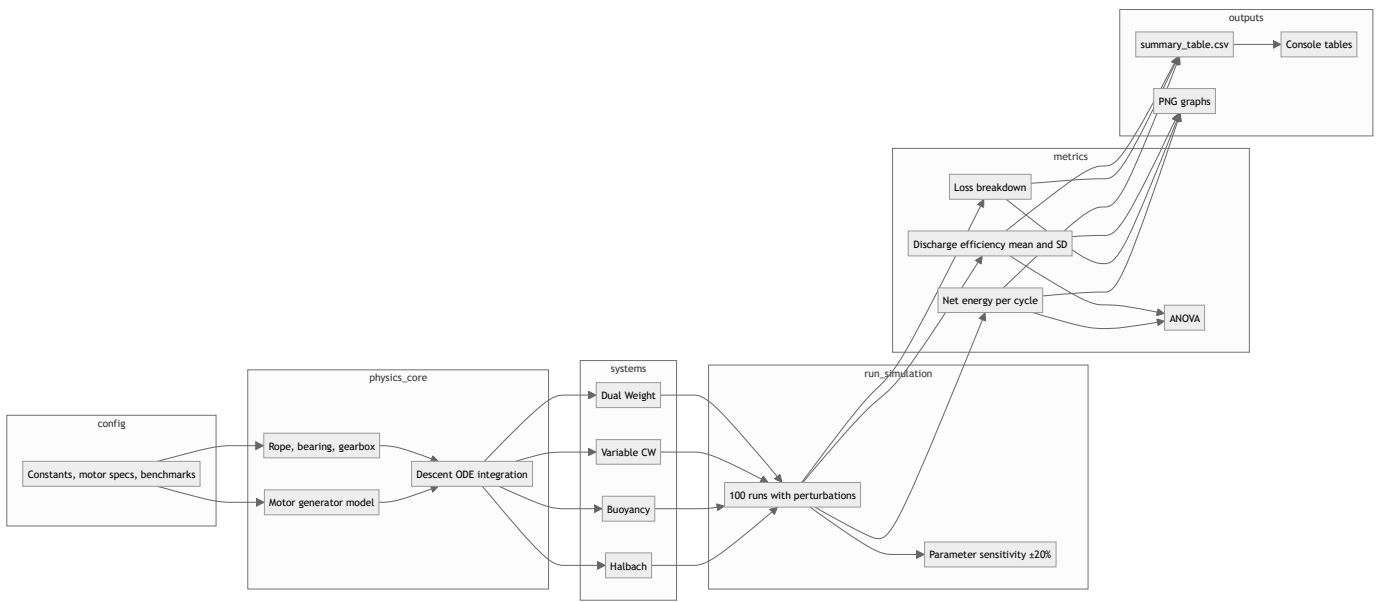
Validation in **calibration.m**, invoked from `main.m`: (1) Energy conservation within 1%. (2) Discharge efficiency 0–100%. (3) Ranking Variable CW > Dual Weight > Buoyancy > Halbach. (4) Physical build calibration vs multimeter (adjust η_{motor} /friction if error > 15%).

7. Module Overview

Step	Component	Dependency
1	config.m: constants, motor parameters	—
2	physics_core.m: rope, bearing, gearbox, motor, ODE	config
3	run_cycle_dual_weight.m, run_cycle_variable_cw.m	physics_core
4	run_cycle_buoyancy.m, run_cycle_halbach.m	physics_core
5	run_simulation.m: 100-run loop, sensitivity	all systems
6	metrics.m: discharge efficiency, loss, net, ANOVA	run_simulation

7	plotting.m: graph sets	metrics
8	calibration.m, main.m	all

8. Diagram: Data and Control Flow



To save as PDF: use your browser's **Print** (Ctrl+P) and choose **Save as PDF** or **Microsoft Print to PDF**.